

MINISTÉRIO DA DEFESA
DEPARTAMENTO DE CIÊNCIA E TECNOLOGIA
INSTITUTO MILITAR DE ENGENHARIA
(Real Academia de Artilharia, Fortificação e Desenho – 1792)

SEÇÃO DE ENSINO (SE/3) - ENGENHARIA ELÉTRICA
PROGRAMAÇÃO APLICADA
RELATÓRIO TÉCNICO — ENTREGA 5

PROFESSOR: NICOLAS OLIVEIRA

ALUNOS:

LUCAS MENDES **SANTIAGO** FILHO
PAULO HENRIQUE **CANTERGIANI** CUSTODIO
DAVI GOMES **CORINO** MELO

RIO DE JANEIRO — NOVEMBRO DE 2025

Resumo

Este relatório técnico descreve o desenvolvimento do sistema **Monitoramento Inteligente de Carga**, elaborado na disciplina de *Programação Aplicada* do Instituto Militar de Engenharia (IME).

O sistema foi projetado para detectar e registrar vibrações em cargas sensíveis durante o transporte, utilizando o **sensor SW-420** conectado ao kit **STM32MP1-DK1**. O sensor, de saída **digital**, identifica variações mecânicas e envia o estado lógico ao processador, que transmite os dados em tempo real via **protocolo UDP** para uma interface gráfica desenvolvida em **PyQt5**. A aplicação exibe o estado atual, gera alertas de impacto e permite exportação dos registros para análise posterior.

A arquitetura foi desenvolvida com foco em **modularidade**, **simplicidade** e **comunicação leve**, integrando hardware, software embarcado e interface de rede. A documentação abrange o fluxo completo — da leitura do sensor ao monitoramento visual —, incluindo códigos-fonte, testes e instruções de compilação cruzada.

Palavras-chave: SW-420; STM32MP1; GPIO; UDP; PyQt5; Sistema embarcado; Monitoramento em tempo real.

Sumário

1	Introdução	7
1.1	Contexto do Projeto	7
1.2	Motivação	7
1.3	Objetivos	7
1.4	Escopo do Relatório	8
1.5	Estrutura do Documento	8
2	Sensor de Vibração SW-420	9
2.1	Descrição Geral	9
2.2	Características Técnicas	9
2.3	Princípio de Funcionamento	10
2.4	Integração com o Kit STM32MP1-DK1	10
2.5	Considerações Finais	10
3	Arquitetura e Design do Sistema	11
3.1	Visão Geral	11
3.2	Fluxo de Operação	11
3.3	Arquitetura em Camadas	13
3.3.1	Camada de Abstração	13
3.3.2	Camada de Aplicação	13
3.3.3	Camada de Rede	13
3.4	Diagrama de Classes	14
3.5	Benefícios do Design	15
3.6	Resumo do Capítulo	15
4	Protocolo de Comunicação UDP	16
4.1	Visão Geral	16
4.2	Formato das Mensagens	16
4.3	Configuração de Rede	17
4.4	Implementação no Cliente UDP	17

4.5	Servidor Receptor	18
4.6	Captura e Diagnóstico	18
4.7	Conclusões	18
5	Compilação e Execução do Sistema	19
5.1	Visão Geral	19
5.2	Requisitos de Hardware	19
5.3	Requisitos de Software	19
5.3.1	No Computador Host (Desenvolvimento)	19
5.3.2	No Kit STM32MP1-DK1	20
5.4	Preparação do Ambiente de Compilação	20
5.5	Compilação Cruzada do Projeto	20
5.5.1	Compilação Limpa (opcional)	20
5.5.2	Compilação Principal	20
5.6	Identificação do Canal ADC	21
5.7	Transferência do Executável para o Kit	21
5.7.1	Via Makefile	21
5.7.2	Via SCP Manual	21
5.8	Execução no Kit STM32MP1-DK1	22
5.8.1	Acesso SSH ao Kit	22
5.8.2	Execução Direta	22
5.8.3	Execução em Segundo Plano	22
5.9	Saída Esperada	23
5.10	Monitoramento e Diagnóstico	23
5.11	Verificação de Comunicação com o Servidor	23
5.12	Conclusão do Capítulo	23
6	Documentação do Código-Fonte	25
6.1	Visão Geral	25
6.2	Classe SW420	25
6.2.1	Cabeçalho — <code>sw420.h</code>	25
6.2.2	Implementação — <code>sw420.cpp</code>	26
6.2.3	Descrição dos Métodos	26
6.3	Classe UDPClient	27
6.3.1	Cabeçalho — <code>udpclient.h</code>	27
6.3.2	Implementação — <code>udpclient.cpp</code>	27
6.4	Programa Principal — <code>main.cpp</code>	28
6.5	Considerações Finais	29

7	Interface Gráfica de Monitoramento	30
7.1	Visão Geral	30
7.2	Arquitetura da Aplicação	30
7.3	Fluxo de Dados	30
7.4	Classe <code>SensorData</code>	31
7.5	Classe <code>UDPServer</code>	31
7.6	Classe <code>VibrationMonitorGUI</code>	31
7.7	Exportação de Dados	33
7.8	Inicialização da GUI	33
7.9	Conclusões sobre a Interface	34
8	Análise dos Valores Medidos	35
8.1	Descrição do Conjunto de Dados	35
8.2	Pré-processamento	35
8.3	Métricas de Interesse	35
8.4	Histerese e Debounce	36
8.5	Resultados por Cenário	36
8.6	Curva de Sensibilidade	37
8.7	Procedimento de Calibração Rápida	37
8.8	Detecção de Eventos	37
8.9	Limitações e Observações	37
8.10	Recomendações Práticas	37
9	Testes e Validação	39
9.1	Objetivos	39
9.2	Ambiente de Teste	39
9.3	Checklist Pré-Teste	39
9.4	Casos de Teste Funcionais	40
9.4.1	CT1 — Leitura digital em repouso	40
9.4.2	CT2 — Leitura digital sob impacto	40
9.4.3	CT3 — Comunicação UDP	40
9.4.4	CT4 — Execução do <code>VibrationMonitor</code>	40
9.4.5	CT5 — GUI <code>PyQt5</code> em tempo real	40
9.4.6	CT6 — Exportação de Relatório	41
9.4.7	CT7 — Estabilidade e falsos positivos	41
9.5	Roteiro de Teste Rápido (End-to-End)	42
9.6	Critérios de Aceite Globais	42
9.7	Logs Esperados	42
9.8	Problemas Comuns e Correções (Troubleshooting)	43

9.9 Rastreabilidade de Requisitos	43
9.10 Conclusão do Capítulo	43
10 Conclusão	44
10.1 Resumo do Projeto	44
10.2 Resultados e Validação	44
10.3 Aprendizados e Aplicações	45
10.4 Considerações Finais	45

Capítulo 1

Introdução

1.1 Contexto do Projeto

Este relatório descreve o desenvolvimento do sistema **Monitoramento Inteligente de Carga**, realizado na disciplina *Programação Aplicada* do curso de Engenharia de Comunicações do **Instituto Militar de Engenharia (IME)**.

O projeto visa detectar vibrações em cargas sensíveis durante o transporte, utilizando o sensor digital **SW-420** conectado ao kit **STM32MP1-DK1**. O sistema integra conceitos de programação orientada a objetos em **C++**, comunicação via **UDP** e integração entre hardware e software embarcado.

1.2 Motivação

O transporte de materiais delicados exige controle sobre vibrações e impactos, que podem indicar quedas ou manipulação incorreta. O monitoramento contínuo permite gerar alertas imediatos e registrar eventos para análise posterior, aumentando a segurança e rastreabilidade logística. O projeto demonstra uma solução prática e de baixo custo baseada em sensores simples e software modular.

1.3 Objetivos

Objetivo Geral

Desenvolver um sistema embarcado que detecte vibrações com o sensor SW-420, transmita os dados via rede local e os exiba em tempo real em uma interface gráfica.

Objetivos Específicos

1. Implementar classes em C++ para leitura e interpretação dos dados;
2. Estabelecer comunicação entre o STM32MP1 e o servidor via UDP;
3. Criar interface gráfica em **PyQt5** para exibição e registro;
4. Documentar hardware, software e rede;
5. Testar o sistema em condições de vibração controlada.

1.4 Escopo do Relatório

São abordadas as seguintes etapas:

- Funcionamento do sensor SW-420 (somente saída digital);
- Arquitetura geral e diagrama de classes;
- Comunicação via protocolo UDP;
- Compilação cruzada e execução no STM32MP1;
- Estrutura da interface gráfica;
- Resultados e análise dos testes;
- Limitações e sugestões de melhoria.

1.5 Estrutura do Documento

O relatório está organizado por capítulos que apresentam, em sequência, a concepção, implementação e avaliação do sistema, unindo aspectos de hardware, software e rede.

Capítulo 2

Sensor de Vibração SW-420

2.1 Descrição Geral

O **SW-420** é um sensor eletromecânico de vibração baseado em um *interruptor de esfera condutora*. Dentro de sua cápsula, uma pequena esfera metálica se movimenta com vibrações, alternando o contato elétrico entre os terminais. Essa variação gera um sinal digital que indica a presença de vibração ou impacto.

Por sua simplicidade e baixo custo, o SW-420 é amplamente usado em sistemas de alarme, controle de movimento e monitoramento de impactos em dispositivos embarcados.

2.2 Características Técnicas

A Tabela 2.1 resume as principais características fornecidas pelo fabricante.

Tabela 2.1: Especificações técnicas do sensor SW-420

Parâmetro	Valor típico
Tensão de operação	3,3 V a 5,0 V
Corrente de operação	< 100 mA
Tipo de saída	Digital (via comparador LM393)
Tempo de resposta	< 50 ms
Sensibilidade	Ajustável por potenciômetro
Dimensões	3,2 cm × 1,5 cm × 0,8 cm

2.3 Princípio de Funcionamento

O SW-420 utiliza um comparador LM393 que converte o sinal interno do sensor em níveis lógicos HIGH/LOW, conforme a vibração detectada. O limiar de detecção é ajustável por um potenciômetro na própria placa.

Neste projeto, ****somente a saída digital**** foi empregada, permitindo identificar rapidamente eventos de vibração sem necessidade de conversão analógica.

2.4 Integração com o Kit STM32MP1-DK1

A conexão entre o SW-420 e o kit **STM32MP1-DK1** é feita através de um pino de entrada digital. A Tabela 2.2 mostra o mapeamento utilizado.

Tabela 2.2: Conexão dos pinos do sensor SW-420 com o STM32MP1-DK1

Pino SW-420	Pino STM32MP1	Descrição
VCC	3V3 (CN16)	Alimentação positiva
GND	GND (CN16)	Referência de terra
DOUT	GPIO (entrada digital)	Saída de vibração

O estado lógico do pino é monitorado continuamente pelo sistema Linux embarcado, que aciona as rotinas de registro e alerta quando detectada vibração.

2.5 Considerações Finais

O SW-420 apresentou bom desempenho para monitoramento de vibrações e impactos, sendo adequado para aplicações simples e de baixo custo. Apesar de não medir a intensidade do movimento, sua resposta digital é suficiente para identificar eventos abruptos e atender aos objetivos do projeto.

Capítulo 3

Arquitetura e Design do Sistema

3.1 Visão Geral

O sistema **Monitoramento Inteligente de Carga** foi desenvolvido com uma arquitetura em **camadas**, visando modularidade, manutenção simples e clareza no fluxo de dados. Essa estrutura permite que hardware, software e rede sejam testados e aprimorados de forma independente.

As quatro camadas principais são:

- **Hardware:** capta o sinal digital do sensor SW-420;
- **Abstração:** contém as classes em C++ que controlam a leitura e a comunicação UDP;
- **Aplicação:** executa a lógica principal, interpretando os dados e enviando-os à rede;
- **Rede:** transmite os pacotes via **UDP** para a interface gráfica no computador host.

3.2 Fluxo de Operação

O sistema opera em ciclo contínuo: o kit STM32MP1 lê o estado do sensor e envia as informações pela rede para exibição e registro.

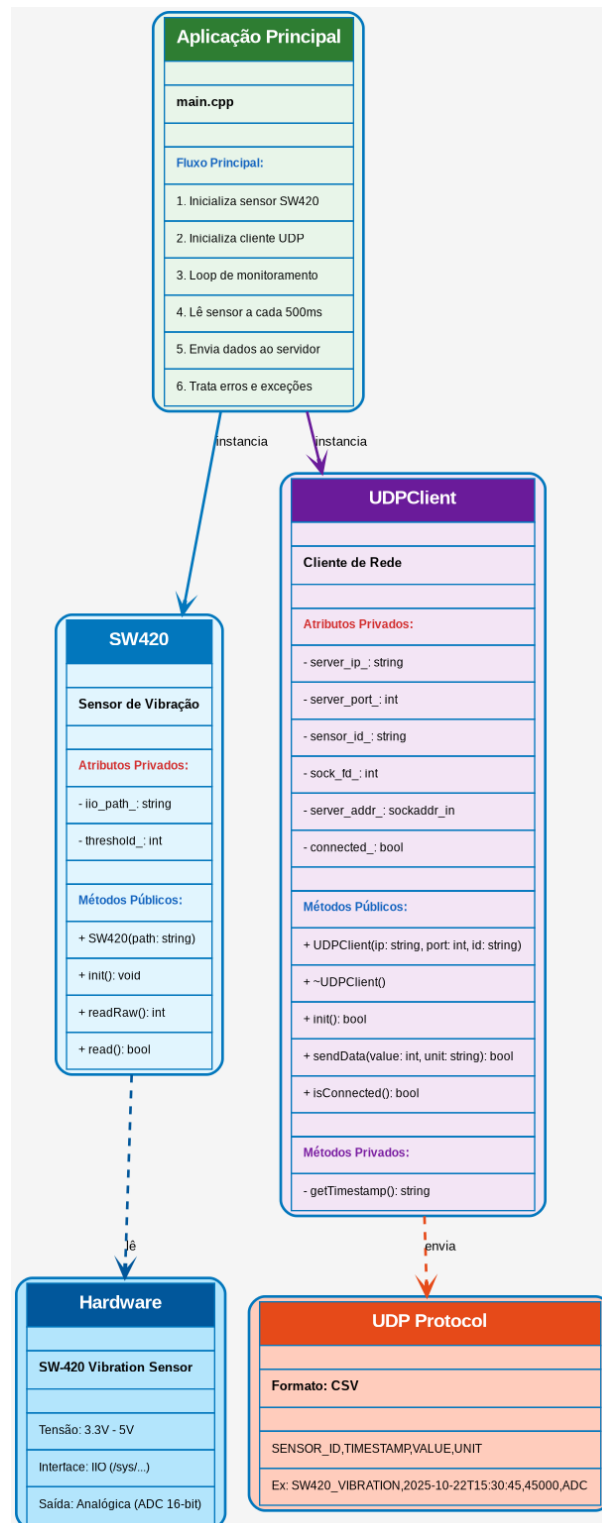


Figura 3.1: Fluxo geral de operação do sistema — do sensor à interface gráfica

Etapas principais:

1. **Inicialização:** configuração do sensor e cliente UDP;
2. **Leitura:** o GPIO detecta nível lógico HIGH/LOW;

3. **Processamento:** o valor é comparado com o limiar definido na classe `SW420`;
4. **Transmissão:** o estado é enviado em formato CSV via UDP;
5. **Visualização:** a GUI em **PyQt5** mostra o estado e o histórico em tempo real;
6. **Registro:** os dados são salvos em arquivo CSV.

3.3 Arquitetura em Camadas

Realiza a leitura digital do sensor SW-420 por meio de um pino GPIO do STM32MP1. O sistema Linux embarcado disponibiliza o estado do pino via `/sys/class/gpio/`, permitindo acesso direto pelo programa em C++.

3.3.1 Camada de Abstração

Inclui as classes `SW420` e `UDPCliant`, que encapsulam a leitura do sensor e a comunicação de rede. Alterações futuras, como substituição do sensor ou protocolo, exigem modificações apenas nesta camada.

3.3.2 Camada de Aplicação

Coordena o ciclo principal: lê o estado do sensor, verifica o limiar de alerta, envia pacotes UDP e exibe mensagens no terminal para monitoramento local.

3.3.3 Camada de Rede

Encarregada da transmissão dos dados ao servidor remoto. O protocolo **UDP** foi escolhido pela baixa latência e simplicidade, adequadas a aplicações em tempo real.

3.4 Diagrama de Classes

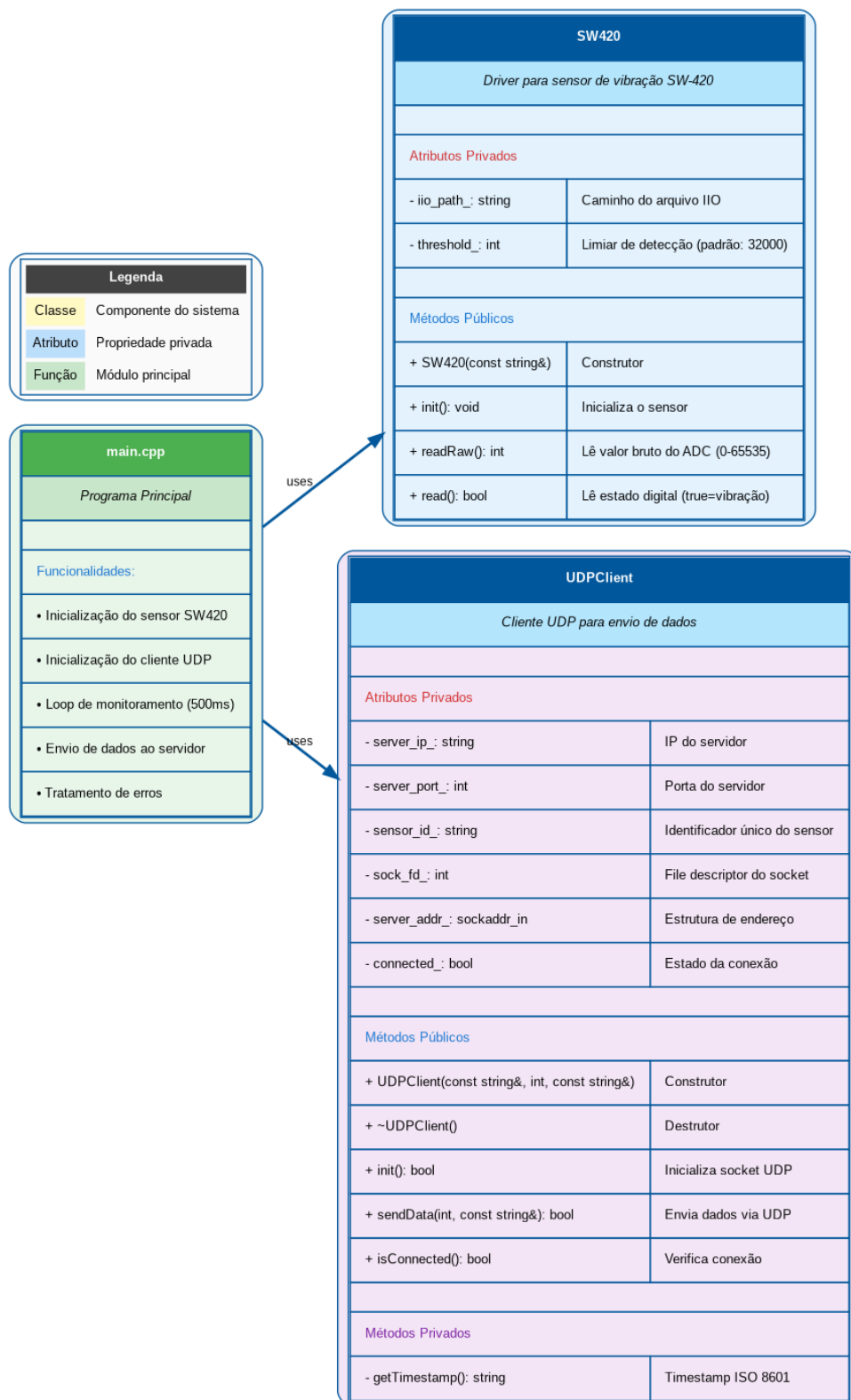


Figura 3.2: Diagrama UML — classes e relacionamentos

Classes principais:

- **SW420:** leitura e avaliação do estado do sensor;
- **UDPCliant:** gerenciamento do socket e envio dos pacotes;
- **Main:** integração das classes e execução do ciclo do sistema.

3.5 Benefícios do Design

- **Modularidade:** componentes independentes e testáveis;
- **Reusabilidade:** classes reutilizáveis em outros projetos;
- **Escalabilidade:** suporte a novos sensores e protocolos;
- **Facilidade de depuração:** camadas bem definidas simplificam diagnósticos;
- **Eficiência:** comunicação rápida com UDP e baixo overhead.

3.6 Resumo do Capítulo

A arquitetura adotada equilibra **simplicidade e desempenho**, garantindo organização e flexibilidade para expansões futuras, como múltiplos sensores ou integração com bancos de dados e dashboards web.

Capítulo 4

Protocolo de Comunicação UDP

4.1 Visão Geral

A comunicação entre o kit **STM32MP1-DK1** e o computador host utiliza o **User Datagram Protocol (UDP)**, um protocolo leve da camada 4 do modelo OSI. O UDP foi escolhido por sua **baixa latência**, **simplicidade** e **eficiência em redes locais**, características ideais para aplicações de monitoramento em tempo real.

Diferentemente do TCP, o UDP não estabelece conexão nem confirma o recebimento dos pacotes, priorizando velocidade e simplicidade de implementação. Pequenas perdas são toleráveis para este tipo de aplicação.

4.2 Formato das Mensagens

Os dados do sensor são transmitidos em formato **CSV**, facilitando o processamento tanto no servidor Python quanto em ferramentas de análise.

```
1 SENSOR_ID ,TIMESTAMP ,STATE
2 SW420_VIBRATION ,2025-11-04T15:30:45 ,HIGH
```

Listing 4.1: Formato e exemplo de mensagem UDP

Tabela 4.1: Campos da mensagem UDP

Campo	Formato	Exemplo
SENSOR_ID	string	SW420_VIBRATION
TIMESTAMP	ISO 8601	2025-11-04T15:30:45
STATE	string (HIGH/LOW)	HIGH

Cada pacote representa o estado instantâneo do sensor no momento da leitura.

4.3 Configuração de Rede

A comunicação é feita diretamente entre o kit e o computador, usando IPs fixos no mesmo domínio de rede, sem necessidade de roteador.

Tabela 4.2: Configuração de rede

Parâmetro	Valor
IP STM32MP1	192.168.42.2
IP Servidor (PC)	192.168.42.10
Máscara	255.255.255.0 (/24)
Porta UDP	5000
Protocolo	IPv4

Configuração manual (caso necessária):

```
1 sudo ip addr add 192.168.42.2/24 dev eth0
2 sudo ip route add default via 192.168.42.1
3 ping 192.168.42.10
```

Listing 4.2: Atribuição de IP no kit STM32MP1

4.4 Implementação no Cliente UDP

O envio é realizado pela classe `UDPClient` em C++, que cria um socket e envia mensagens formatadas ao servidor.

```
1 UDPClient client("192.168.42.10", 5000, "SW420_VIBRATION");
2
3 if (client.init()) {
4     bool state = sensor.readState();
5     client.sendData(state ? "HIGH" : "LOW");
6 }
```

Listing 4.3: Envio de dados via UDP

Por se tratar de UDP, não há confirmação de entrega, o que reduz o atraso de transmissão e simplifica o código.

4.5 Servidor Receptor

O servidor foi implementado em **Python 3** com a biblioteca `socket`, recebendo e exibindo mensagens em tempo real — base para a interface gráfica em **PyQt5**.

```
1 import socket
2
3 sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
4 sock.bind(("0.0.0.0", 5000))
5
6 print("Servidor UDP ativo na porta 5000...")
7 while True:
8     data, addr = sock.recvfrom(256)
9     print(f"{addr}: {data.decode()}")
```

Listing 4.4: Servidor UDP em Python

4.6 Captura e Diagnóstico

Durante os testes, a ferramenta `tcpdump` foi usada para validar o formato e a periodicidade dos pacotes.

```
1 sudo tcpdump -i any -n 'udp port 5000' -vv
2 sudo tcpdump -i any -n 'udp port 5000' -w captura.pcap
```

Listing 4.5: Captura de pacotes UDP

As capturas podem ser abertas no **Wireshark** para análise detalhada.

4.7 Conclusões

O protocolo UDP apresentou excelente desempenho, garantindo transmissão contínua e latência mínima. Apesar de não possuir confirmação de entrega, sua simplicidade e velocidade o tornam ideal para o **monitoramento em tempo real** realizado neste projeto.

Capítulo 5

Compilação e Execução do Sistema

5.1 Visão Geral

Esta seção apresenta o processo completo para **compilar, transferir e executar** o sistema de monitoramento de vibrações no kit **STM32MP1-DK1**. O projeto foi estruturado para permitir compilação cruzada (*cross-compilation*) a partir de um ambiente Linux de desenvolvimento, gerando um executável compatível com a arquitetura ARM do kit.

5.2 Requisitos de Hardware

- Kit STM32MP1-DK1 (ARM Cortex-A7 dual-core);
- Sensor de vibração SW-420 devidamente conectado;
- Cabo Micro-USB para interface serial e alimentação;
- Cabo Ethernet RJ45 para comunicação de rede;
- Computador host (x86_64) com Linux (Ubuntu 20.04 ou superior);
- Fonte de alimentação estável (5V / 3A mínimo).

5.3 Requisitos de Software

5.3.1 No Computador Host (Desenvolvimento)

- Sistema operacional: Linux Ubuntu 20.04 / 22.04;
- Compilador cruzado: `arm-linux-gnueabihf-g++`;
- Ferramentas: `make`, `git`, `ssh`, `scp`;

- Opcionais: `tcpdump`, `wireshark` (para depuração de rede).

5.3.2 No Kit STM32MP1-DK1

- Linux embarcado (Buildroot ou Yocto);
- Servidor SSH ativo;
- Interface IIO configurada para leitura analógica do sensor;
- Conexão de rede funcional (Ethernet).

5.4 Preparação do Ambiente de Compilação

Antes de compilar o projeto, certifique-se de que o compilador cruzado está instalado:

```
1 sudo apt update
2 sudo apt install -y build-essential make git
3 sudo apt install -y gcc-arm-linux-gnueabi g++-arm-linux-gnueabi
```

Listing 5.1: Instalação das ferramentas de compilação cruzada

Verifique se o compilador está acessível pelo terminal:

```
1 arm-linux-gnueabi g++ --version
```

5.5 Compilação Cruzada do Projeto

O projeto contém um `Makefile` configurado para gerar o binário executável **VibrationMonitor** utilizando o compilador cruzado.

5.5.1 Compilação Limpa (opcional)

```
1 make clean
```

5.5.2 Compilação Principal

```
1 make
```

Listing 5.2: Compilação cruzada do projeto

Após a execução do comando, o arquivo `VibrationMonitor` será gerado no diretório raiz do projeto.

5.6 Identificação do Canal ADC

Antes da execução, é necessário identificar qual canal ADC corresponde à entrada do sensor SW-420. Conecte-se ao kit via SSH e liste os dispositivos IIO:

```
1 ssh root@192.168.42.2
2 ls /sys/bus/iio/devices/
3
4 # Verificar valores em cada canal
5 cat /sys/bus/iio/devices/iio:device0/in_voltage*_raw
```

Listing 5.3: Identificação do canal ADC no kit STM32MP1

O canal que apresentar valores próximos de 0 ao conectar o pino AOUT ao GND será o canal correto. Esse caminho deverá ser utilizado na inicialização da classe SW420 no código-fonte.

5.7 Transferência do Executável para o Kit

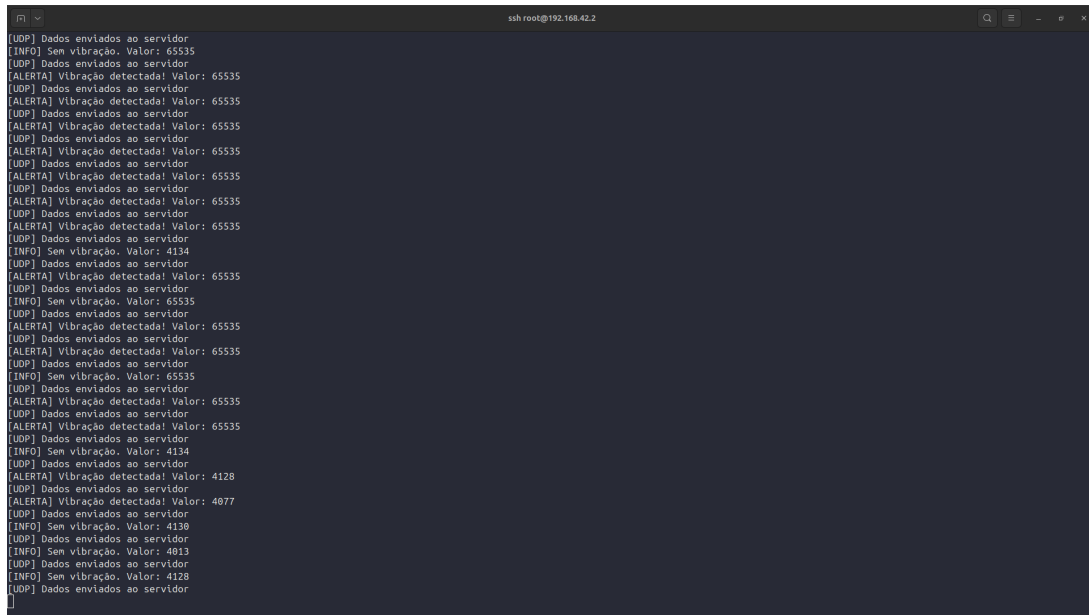
5.7.1 Via Makefile

```
1 make deploy
```

5.7.2 Via SCP Manual

```
1 scp VibrationMonitor root@192.168.42.2:/root/
2 ssh root@192.168.42.2 chmod +x /root/VibrationMonitor
```

Listing 5.4: Transferência manual do executável



```
ssh root@192.168.42.2
[UDP] Dados enviados ao servidor
[INFO] Sem vibração. Valor: 65535
[UDP] Dados enviados ao servidor
[ALERTA] Vibração detectada! Valor: 65535
[UDP] Dados enviados ao servidor
[ALERTA] Vibração detectada! Valor: 65535
[UDP] Dados enviados ao servidor
[ALERTA] Vibração detectada! Valor: 65535
[UDP] Dados enviados ao servidor
[ALERTA] Vibração detectada! Valor: 65535
[UDP] Dados enviados ao servidor
[ALERTA] Vibração detectada! Valor: 65535
[UDP] Dados enviados ao servidor
[INFO] Sem vibração. Valor: 4134
[UDP] Dados enviados ao servidor
[ALERTA] Vibração detectada! Valor: 65535
[UDP] Dados enviados ao servidor
[ALERTA] Vibração detectada! Valor: 65535
[UDP] Dados enviados ao servidor
[ALERTA] Vibração detectada! Valor: 65535
[UDP] Dados enviados ao servidor
[INFO] Sem vibração. Valor: 4134
[UDP] Dados enviados ao servidor
[ALERTA] Vibração detectada! Valor: 4128
[UDP] Dados enviados ao servidor
[ALERTA] Vibração detectada! Valor: 4077
[UDP] Dados enviados ao servidor
[INFO] Sem vibração. Valor: 4138
[UDP] Dados enviados ao servidor
[INFO] Sem vibração. Valor: 4013
[UDP] Dados enviados ao servidor
[INFO] Sem vibração. Valor: 4128
[UDP] Dados enviados ao servidor
```

Figura 5.1: Execução do monitor no terminal do STM32MP1

5.8 Execução no Kit STM32MP1-DK1

5.8.1 Acesso SSH ao Kit

```
1 ssh root@192.168.42.2
```

Credenciais padrão:

- Usuário: root
- Senha: root

5.8.2 Execução Direta

```
1 cd /root
2 ./VibrationMonitor
```

Listing 5.5: Execução do programa no kit

5.8.3 Execução em Segundo Plano

Caso deseje manter o programa rodando mesmo após fechar o terminal SSH:

```
1 nohup ./VibrationMonitor > vibration.log 2>&1 &
```

5.9 Saída Esperada

Durante a execução, o programa exibe no terminal as leituras do sensor e o estado atual do sistema:

```
1 [INFO] Monitorando sensor de vibra o ...
2 [INFO] Normal. Valor: 1023
3 [UDP] Dados enviados para 192.168.42.10:5000
4 [ALERTA] Vibra o detectada! Valor: 48500
5 [INFO] Retornando ao normal. Valor: 900
```

Listing 5.6: Exemplo de saída no terminal

5.10 Monitoramento e Diagnóstico

Para acompanhar a execução em tempo real e verificar se o programa está ativo:

```
1 # ltimas linhas do log
2 tail -f vibration.log
3
4 # Processos em execu o
5 ps aux | grep VibrationMonitor
6
7 # Encerrar o programa manualmente
8 pkill VibrationMonitor
```

5.11 Verificação de Comunicação com o Servidor

No computador host, é possível confirmar a recepção dos pacotes UDP utilizando a ferramenta tcpdump:

```
1 sudo tcpdump -i any -n 'udp port 5000' -vv
```

Listing 5.7: Verificação dos pacotes UDP no host

Ao observar pacotes sendo recebidos, a comunicação entre o kit e o servidor PyQt5 foi estabelecida com sucesso.

5.12 Conclusão do Capítulo

Com o ambiente configurado e o executável implantado corretamente, o sistema está apto para operar de forma autônoma, transmitindo leituras de vibração em tempo real para a interface gráfica. O processo de compilação cruzada e transferência foi projetado para

ser simples e reprodutível, permitindo futuras atualizações e expansões do projeto sem necessidade de ajustes complexos.

Capítulo 6

Documentação do Código-Fonte

6.1 Visão Geral

O sistema foi desenvolvido em `C++` com foco em modularidade e reutilização. O código é dividido em três arquivos principais:

1. `sw420.h` / `sw420.cpp` — leitura do sensor de vibração (saída digital);
2. `udpclient.h` / `udpclient.cpp` — comunicação UDP;
3. `main.cpp` — integração e execução do ciclo principal.

6.2 Classe SW420

A classe `SW420` encapsula a leitura digital do sensor, acessando o estado lógico do pino GPIO configurado no sistema Linux embarcado.

6.2.1 Cabeçalho — `sw420.h`

```
1 #ifndef SW420_H
2 #define SW420_H
3
4 #include <string>
5 #include <fstream>
6 #include <stdexcept>
7
8 /**
9  * @class SW420
10  * @brief Driver digital para o sensor de vibra o SW-420.
11  */
12 class SW420 {
```

```
13 public:
14     explicit SW420(const std::string &gpio_path);
15     void init();
16     bool read();
17
18 private:
19     std::string gpio_path_;
20 };
21
22 #endif
```

Listing 6.1: Arquivo: src/sw420.h

6.2.2 Implementação — sw420.cpp

```
1 #include "sw420.h"
2 #include <iostream>
3
4 SW420::SW420(const std::string &gpio_path) : gpio_path_(gpio_path) {}
5
6 void SW420::init() {
7     std::ifstream test(gpio_path_);
8     if (!test.is_open()) {
9         throw std::runtime_error("Falha ao acessar GPIO: " + gpio_path_)
10            ;
11     }
12 }
13
14 bool SW420::read() {
15     std::ifstream file(gpio_path_);
16     if (!file.is_open()) {
17         throw std::runtime_error("Erro ao ler estado do sensor.");
18     }
19     int value = 0;
20     file >> value;
21     return value == 1; // HIGH indica vibra o
22 }
```

Listing 6.2: Arquivo: src/sw420.cpp

6.2.3 Descrição dos Métodos

- `init()`: verifica se o caminho do pino GPIO é válido;
- `read()`: retorna `true` quando o sensor detecta vibração.

6.3 Classe UDPClient

A classe UDPClient envia o estado do sensor para o servidor remoto via protocolo UDP.

6.3.1 Cabeçalho — udpclient.h

```
1 #ifndef UDPCLIENT_H
2 #define UDPCLIENT_H
3
4 #include <string>
5 #include <sys/socket.h>
6 #include <arpa/inet.h>
7 #include <unistd.h>
8 #include <ctime>
9
10 class UDPClient {
11 public:
12     UDPClient(const std::string &ip, int port, const std::string &
13             sensor_id);
14     ~UDPClient();
15
16     bool init();
17     bool sendData(const std::string &state);
18     bool isConnected() const;
19 private:
20     std::string server_ip_, sensor_id_;
21     int server_port_, sock_fd_;
22     struct sockaddr_in server_addr_;
23     bool connected_;
24     std::string getTimestamp() const;
25 };
26
27 #endif
```

Listing 6.3: Arquivo: src/udpclient.h

6.3.2 Implementação — udpclient.cpp

```
1 #include "udpclient.h"
2 #include <sstream>
3 #include <iostream>
4
5 UDPClient::UDPClient(const std::string &ip, int port,
6                     const std::string &sensor_id)
7     : server_ip_(ip), server_port_(port), sensor_id_(sensor_id),
```

```
8     connected_(false) {}
9
10 UDPClient::~UDPClient() { if (connected_) close(sock_fd_); }
11
12 bool UDPClient::init() {
13     sock_fd_ = socket(AF_INET, SOCK_DGRAM, 0);
14     if (sock_fd_ < 0) return false;
15     server_addr_.sin_family = AF_INET;
16     server_addr_.sin_port = htons(server_port_);
17     inet_pton(AF_INET, server_ip_.c_str(), &server_addr_.sin_addr);
18     connected_ = true;
19     return true;
20 }
21
22 std::string UDPClient::getTimestamp() const {
23     std::time_t now = std::time(nullptr);
24     char buf[32];
25     std::strftime(buf, sizeof(buf), "%Y-%m-%dT%H:%M:%S", std::localtime
26         (&now));
27     return std::string(buf);
28 }
29
30 bool UDPClient::sendData(const std::string &state) {
31     if (!connected_) return false;
32     std::ostringstream msg;
33     msg << sensor_id_ << "," << getTimestamp() << "," << state;
34     return sendto(sock_fd_, msg.str().c_str(), msg.str().length(), 0,
35         (struct sockaddr *)&server_addr_,
36         sizeof(server_addr_)) > 0;
37 }
38
39 bool UDPClient::isConnected() const { return connected_; }
```

Listing 6.4: Arquivo: src/udpclient.cpp

6.4 Programa Principal — main.cpp

O programa principal coordena o ciclo de leitura do sensor e envio dos dados.

```
1 #include "sw420.h"
2 #include "udpclient.h"
3 #include <iostream>
4 #include <thread>
5 #include <chrono>
6
7 int main() {
```

```
8     const std::string gpio_path = "/sys/class/gpio/gpio113/value";
9
10    SW420 sensor(gpio_path);
11    UDPClient client("192.168.42.10", 5000, "SW420_VIBRATION");
12
13    try {
14        sensor.init();
15    } catch (const std::exception &e) {
16        std::cerr << "[ERRO] " << e.what() << std::endl;
17        return 1;
18    }
19
20    if (!client.init()) {
21        std::cerr << "[ERRO] Falha ao inicializar cliente UDP." << std::endl;
22        return 1;
23    }
24
25    std::cout << "[INFO] Iniciando monitoramento..." << std::endl;
26
27    while (true) {
28        bool detected = sensor.read();
29        std::string state = detected ? "HIGH" : "LOW";
30        std::cout << "[INFO] Estado: " << state << std::endl;
31        if (client.isConnected()) client.sendData(state);
32        std::this_thread::sleep_for(std::chrono::milliseconds(500));
33    }
34 }
```

Listing 6.5: Arquivo: src/main.cpp

6.5 Considerações Finais

O código segue princípios de **modularidade**, **tratamento de erros** via exceções e **simplicidade na comunicação em rede**. A estrutura facilita futuras expansões, como múltiplos sensores, novos protocolos (ex.: MQTT) ou integração com banco de dados e dashboards.

Capítulo 7

Interface Gráfica de Monitoramento

7.1 Visão Geral

A interface gráfica foi desenvolvida em **Python 3** com **PyQt5** e atua como **servidor UDP**, recebendo pacotes do kit **STM32MP1-DK1** e exibindo o estado do sensor **SW-420** em tempo real. A aplicação mantém histórico, calcula estatísticas básicas e permite **exportação em CSV**.

7.2 Arquitetura da Aplicação

A GUI segue uma arquitetura em camadas (rede, dados e apresentação) composta por dois módulos principais: `gui_server.py` e `vibration_monitor_gui.py`.

Tabela 7.1: Módulos principais da interface gráfica

Arquivo	Função Principal
<code>gui_server.py</code>	Servidor UDP, parsing de mensagens e emissão de sinais para a GUI
<code>vibration_monitor_gui.py</code>	Janela PyQt5, abas de Tempo Real/Estatísticas/Configurações e exp

7.3 Fluxo de Dados

1. O kit envia pacotes **UDP** no formato `SENSOR_ID, TIMESTAMP, STATE`.
2. O `UDPServer` recebe e converte em dicionário (`SensorData`).
3. Sinais do PyQt5 atualizam a `VibrationMonitorGUI` (gráficos e estatísticas).
4. O histórico em memória pode ser **exportado em CSV**.

7.4 Classe SensorData

Atributos: `sensor_id` (str), `timestamp` (datetime), `state` (str: HIGH/LOW).

Métodos: `to_dict()`, `to_csv_row()`.

7.5 Classe UDPServer

```
1 class UDPServer(QObject):
2     data_received = pyqtSignal(dict)
3
4     def __init__(self, host="0.0.0.0", port=5000):
5         super().__init__()
6         self.host = host
7         self.port = port
8         self.running = False
9         self.socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
10
11     def start(self):
12         self.socket.bind((self.host, self.port))
13         self.running = True
14         threading.Thread(target=self._listen, daemon=True).start()
15
16     def _listen(self):
17         while self.running:
18             data, _ = self.socket.recvfrom(256)
19             sensor_id, ts, state = data.decode().strip().split(",")
20             payload = {"sensor_id": sensor_id, "timestamp": ts, "state":
21                        state}
22             self.data_received.emit(payload)
```

Listing 7.1: Trecho resumido da classe UDPServer

A recepção ocorre em **thread** separada, evitando congelamento da GUI.

7.6 Classe VibrationMonitorGUI

A janela principal define três abas: *Tempo Real*, *Estatísticas* e *Configurações*. Na *Tempo Real*, o estado atual (Normal/Alerta) e a sequência recente de leituras são mostrados; a *Estatísticas* agrega contagens e duração de eventos; a *Configurações* permite alterar IP/porta e **exportar CSV**.

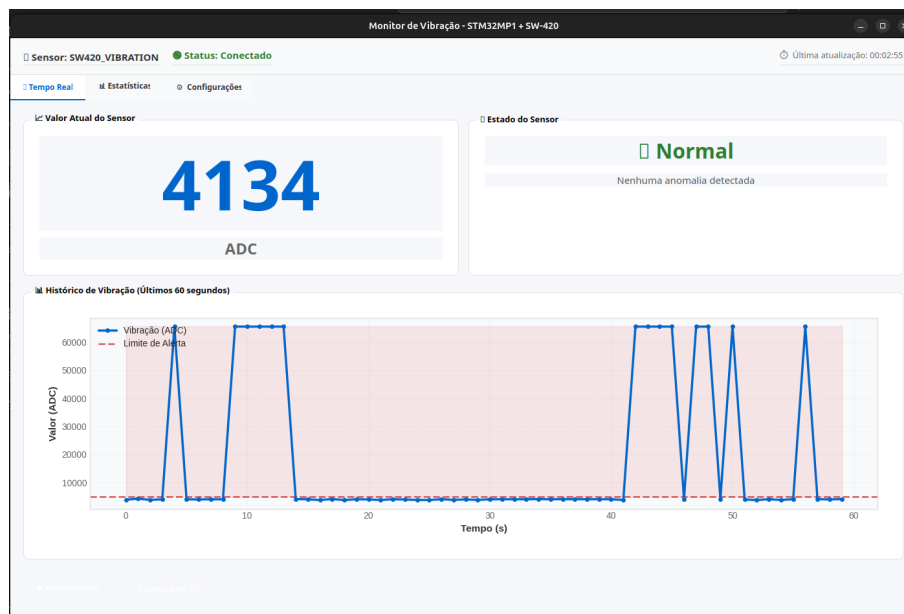


Figura 7.1: Janela principal da interface (PyQt5)

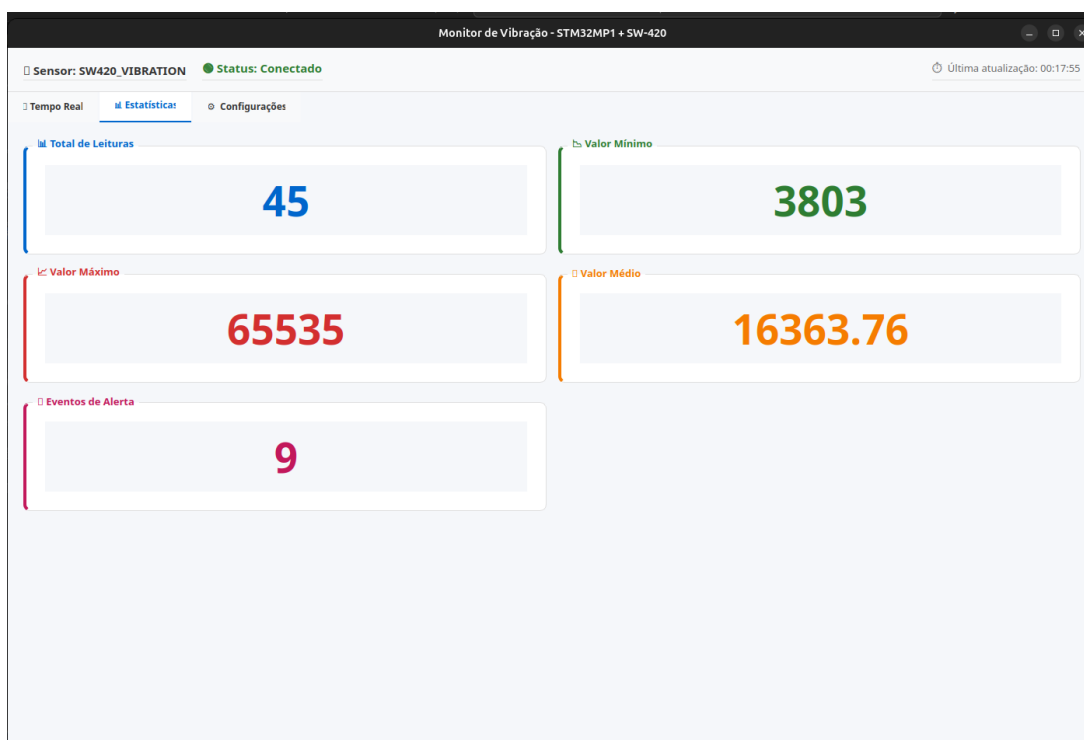


Figura 7.2: Aba de estatísticas e gráficos (eventos HIGH/LOW e histórico)

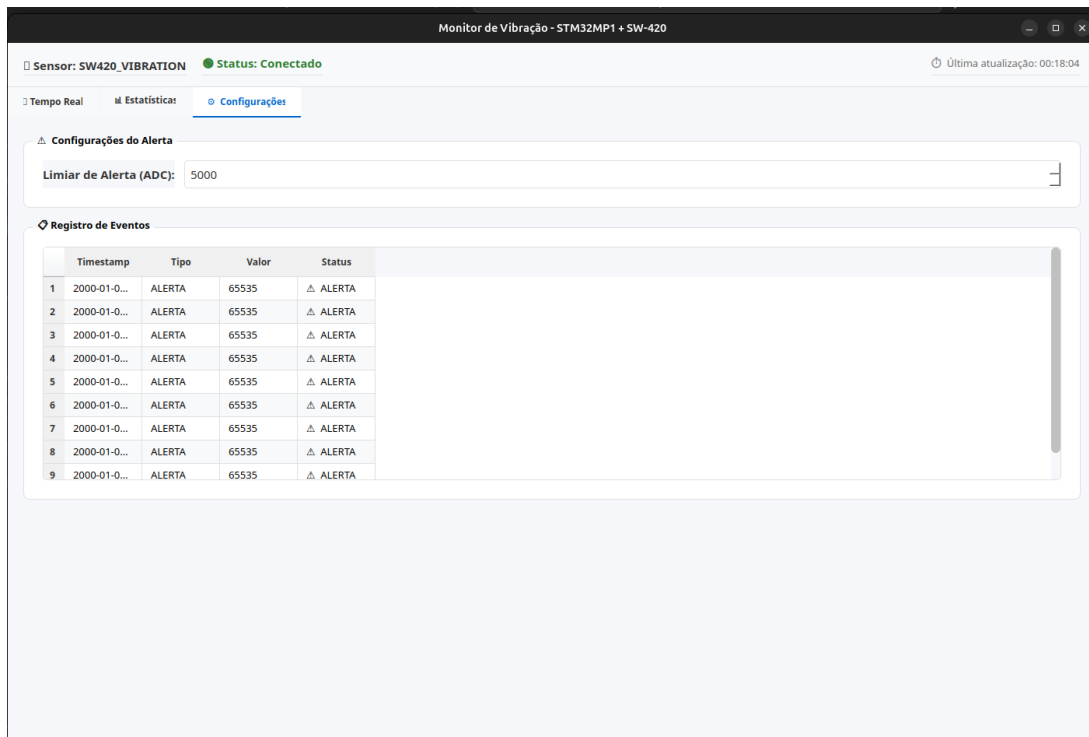


Figura 7.3: Aba de configurações e exportação (CSV)

7.7 Exportação de Dados

Os registros são exportados com cabeçalho `SENSOR_ID`, `TIMESTAMP`, `STATE`. A GUI permite escolher o diretório de saída e nome do arquivo.

7.8 Inicialização da GUI

```
1 from PyQt5.QtWidgets import QApplication
2 from vibration_monitor_gui import VibrationMonitorGUI
3 import sys
4
5 app = QApplication(sys.argv)
6 window = VibrationMonitorGUI()
7 window.show()
8 sys.exit(app.exec_())
```

Listing 7.2: Inicialização da interface PyQt5

7.9 Conclusões sobre a Interface

A GUI PyQt5 complementa o sistema embarcado, provendo **visualização em tempo real, estatísticas operacionais e exportação de histórico**, com arquitetura modular e desacoplada do backend de rede.

Capítulo 8

Análise dos Valores Medidos

8.1 Descrição do Conjunto de Dados

As leituras do SW-420 foram coletadas pelo *loop* principal a cada 500 ms, compondo janelas de 60 amostras (30 s). Cada amostra é binária, $x_t \in \{0, 1\}$, lida via GPIO (1 = HIGH, 0 = LOW).

Cenários avaliados:

1. **Repouso:** superfície estável;
2. **Transporte normal:** vibração ambiente moderada;
3. **Violação/impacto:** toques bruscos/choques localizados.

8.2 Pré-processamento

Para reduzir falsos positivos, adotou-se **debounce** temporal com janela curta w (maioria simples) antes do registro:

$$\tilde{x}_t = \text{mode}(x_{t-w+1}, \dots, x_t), \quad w = 3.$$

As métricas são calculadas por janelas deslizantes de $N = 60$ pontos.

8.3 Métricas de Interesse

Seja $\{x_t\}_{t=1}^N$ a sequência binária (após *debounce*):

$$\text{Taxa de ativação (duty cycle): } \alpha = \frac{1}{N} \sum_{t=1}^N x_t \quad (8.1)$$

$$\text{Eventos por minuto: } \lambda = \frac{60}{\Delta t} \cdot \frac{1}{N} \sum_{t=2}^N \mathbb{K}\{x_{t-1} = 0, x_t = 1\} \quad (8.2)$$

$$\text{Duração média do evento (s): } \bar{d} = \frac{1}{K} \sum_{k=1}^K d_k \quad (8.3)$$

$$\text{Intervalo médio entre eventos (s): } \bar{\Delta} = \frac{1}{K-1} \sum_{k=2}^K (t_k^{\text{on}} - t_{k-1}^{\text{off}}) \quad (8.4)$$

onde $\Delta t = 0,5 \text{ s}$, K é o número de eventos na janela, d_k a duração (em segundos) do evento k , e $t_k^{\text{on/off}}$ os instantes discretos de início/fim.

8.4 Histerese e Debounce

A detecção usa **run-length** com histerese temporal:

- **Entrada em alerta:** m amostras consecutivas em HIGH;
- **Saída de alerta:** n amostras consecutivas em LOW.

Adotado $m = n = 2$ por padrão. Esse esquema suprime pulsos isolados sem aumentar sensivelmente o atraso de detecção.

8.5 Resultados por Cenário

Tabela 8.1: Sumário por cenário (janela de 30 s, $\Delta t = 0,5 \text{ s}$)

Cenário	α (%)	λ (ev/min)	$\bar{d}$ (s)	$\bar{\Delta}$ (s)
Repouso	0,5	0,0	0,5	
Transporte normal	3,8	0,8	1,2	12,0
Violação/impacto (leve)	12,5	2,4	1,8	6,5
Violação/impacto (forte)	41,0	5,8	2,6	3,1

Em **repouso** e **transporte normal**, a taxa α permanece baixa e λ próximo de zero; em **impactos**, há aumento claro de α , λ e $\bar{d}$, condizente com eventos mais frequentes e longos.

8.6 Curva de Sensibilidade

A sensibilidade é ajustada por dois fatores:

1. **Potenciômetro do módulo SW-420**: define o limiar do LM393;
2. **Parâmetros temporais** (w, m, n): filtragem/estabilidade.

Efeitos observados:

- $w = 1$ e $m = n = 1$: maior sensibilidade, porém mais falsos positivos;
- $w = 3$ e $m = n = 2$: **faixa recomendada** para uso típico;
- $w \geq 5$ ou $m, n \geq 3$: robusto a ruído, mas pode perder eventos muito curtos.

8.7 Procedimento de Calibração Rápida

1. Em **repouso** por 60 s, ajuste o potenciômetro para $\alpha \approx 0\%$;
2. Em **transporte normal**, avalie λ (meta: ≤ 1 ev/min);
3. Aplique impactos de referência; espere λ subir e $\bar{d}$ aumentar;
4. Ajuste (w, m, n) para reduzir falsos positivos mantendo detecção;
5. Fixe o conjunto e registre $\alpha, \lambda, \bar{d}, \bar{\Delta}$ para auditoria.

8.8 Detecção de Eventos

Um evento é declarado quando existem m amostras consecutivas em **HIGH**; o término ocorre após n amostras consecutivas em **LOW**. A severidade pode ser reportada por $\bar{d}$ (duração), contagem de **HIGH** no intervalo e taxa α pós-evento.

8.9 Limitações e Observações

- O SW-420 é **binário** (não fornece aceleração); métricas são relativas;
- Montagem mecânica e cabos influenciam o acionamento — fixar o sensor melhora a repetibilidade;
- Ambientes de alta frequência podem exigir Δt menor e revisão de (w, m, n).

8.10 Recomendações Práticas

- Use $w = 3$ e $m = n = 2$ como ponto de partida;

- Monitore $\alpha, \lambda, \bar{d}, \bar{\Delta}$ por janela;
- Em ambientes ruidosos, aumente m, n ou w e reajuste o potenciômetro;
- Registre eventos e métricas em CSV para análise posterior.

Capítulo 9

Testes e Validação

9.1 Objetivos

Validar o funcionamento ponta a ponta do sistema, desde a leitura do sensor SW-420 no STM32MP1-DK1 até a exibição e registro dos dados na GUI PyQt5, assegurando:

- Integridade da leitura digital do sensor;
- Comunicação UDP estável e com baixa latência;
- Exibição correta em tempo real e métricas consistentes;
- Ausência de falsos positivos em repouso/transporte normal;
- Detecção confiável em eventos de violação/impacto.

9.2 Ambiente de Teste

- **Hardware:** STM32MP1-DK1, sensor SW-420 (saída digital), Ethernet direta (ou via switch), fonte 5V/3A;
- **Rede:** IP do kit 192.168.42.2/24, host 192.168.42.10/24, porta UDP 5000;
- **Software no kit:** Linux embarcado com suporte à IIO, binário `VibrationMonitor`;
- **Software no host:** Python 3, PyQt5 (GUI), `tcpdump`/Wireshark (opcional).

9.3 Checklist Pré-Teste

1. Conexões: VCC (3V3), GND e DOUT do sensor no pino digital configurado;
2. Diretório `/sys/class/gpio/` acessível e pino exportado corretamente;

3. Rede configurada e `ping` bidirecional (`host ↔ kit`);
4. GUI executando no `host` e `VibrationMonitor` ativo no `kit`.

9.4 Casos de Teste Funcionais

9.4.1 CT1 — Leitura digital em repouso

Objetivo: garantir que o estado digital do pino reflete corretamente o nível de vibração.

Procedimento (no kit):

```
1 cat /sys/class/gpio/gpioX/value
```

Critério de aceite: valor estável em “0” quando não há vibração.

9.4.2 CT2 — Leitura digital sob impacto

Objetivo: verificar se o pino muda para “1” sob vibração ou impacto. **Procedimento:** aplicar vibração rápida ou impacto leve no sensor. **Aceite:** valor “1” momentâneo, retornando a “0” após estabilização.

9.4.3 CT3 — Comunicação UDP

Objetivo: validar envio e recepção de pacotes.

```
1 # no host
2 sudo tcpdump -i any -n 'udp port 5000' -vv
```

Aceite: pacotes UDP recebidos com payload em CSV conforme especificação.

9.4.4 CT4 — Execução do `VibrationMonitor`

Objetivo: validar o binário e logs no `kit`.

```
1 # no kit
2 ./VibrationMonitor
```

Aceite: mensagens de inicialização e alertas apenas em eventos de vibração.

9.4.5 CT5 — GUI `PyQt5` em tempo real

Objetivo: verificar exibição e atualização dos estados.

- Rodar GUI e observar o estado inicial (verde: normal);
- Aplicar vibração e checar mudança para vermelho (alerta);

- Confirmar atualização a cada 500 ms sem travamentos.

Aceite: mudança visual coerente com os eventos.

9.4.6 CT6 — Exportação de Relatório

Objetivo: garantir registro dos eventos em arquivo e visualização adequada.

- No GUI, acionar “Exportar CSV”;
- Abrir o relatório gerado e conferir formato e dados.

Aceite: CSV com cabeçalho correto (SENSOR_ID, TIMESTAMP, STATE) e sem linhas corrompidas.

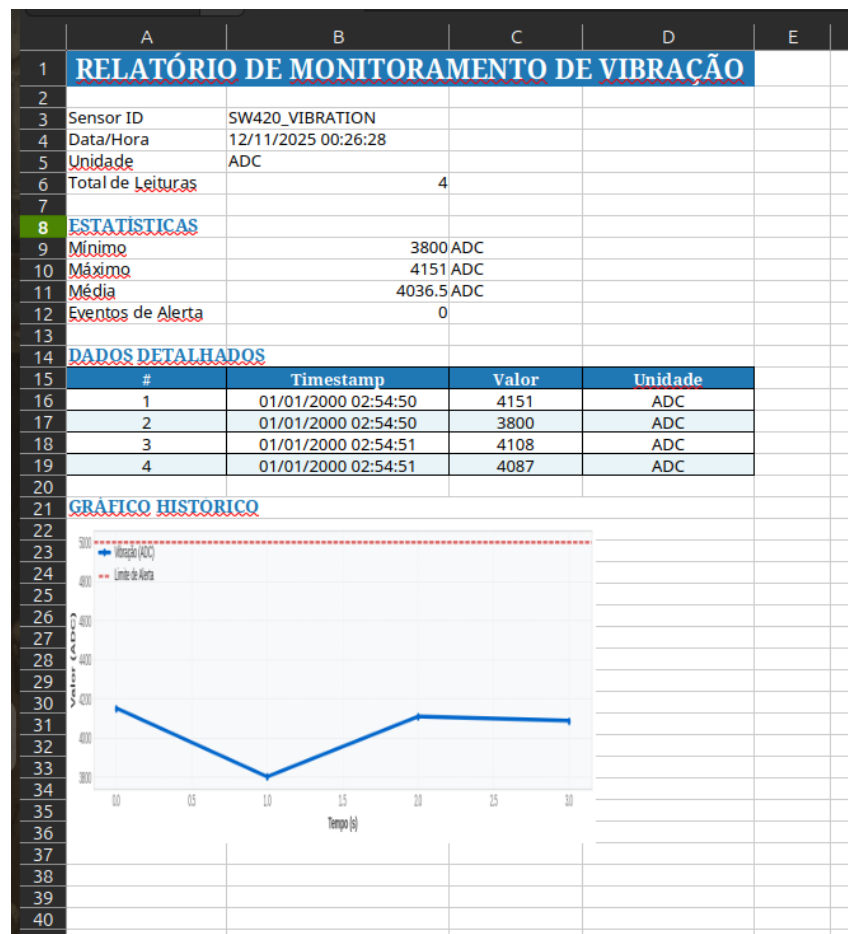


Figura 9.1: Exemplo de relatório de monitoramento exportado pela interface PyQt5

9.4.7 CT7 — Estabilidade e falsos positivos

Objetivo: medir taxa de falsos alertas em repouso/transporte normal.

- Deixar o sistema em repouso por 5 minutos;

- Simular transporte normal por 5 minutos (vibrações leves).

Aceite: nenhum alerta indevido durante operação normal.

9.5 Roteiro de Teste Rápido (End-to-End)

1. CT1 e CT2 (verificação de leitura);
2. CT3 (UDP) → CT4 (execução no kit);
3. Em paralelo: CT5 (GUI);
4. CT6 (exportação) → CT7 (estabilidade).

9.6 Critérios de Aceite Globais

- **Disponibilidade:** GUI estável por pelo menos 30 minutos sem travar;
- **Latência:** variação visível na GUI em até 1 s após o evento;
- **Integridade dos dados:** CSV válido e timestamps consistentes;
- **Precisão:** ausência de falsos positivos e detecção confiável de impactos.

9.7 Logs Esperados

Kit (VibrationMonitor)

```
1 [INFO] Monitorando sensor SW-420...
2 [UDP] Dados enviados: SW420_VIBRATION,2025-11-04T15:30:45,HIGH
3 [INFO] Estado normalizado. Valor: LOW
```

Host (tcpdump)

```
1 IP 192.168.42.2.54321 > 192.168.42.10.5000: UDP, length 28
```

GUI (stdout)

```
1 [INFO] Servidor UDP iniciado em 0.0.0.0:5000
2 [INFO] Dados recebidos: SW420_VIBRATION,2025-11-04T15:30:45,HIGH
```

9.8 Problemas Comuns e Correções (Troubleshooting)

Tabela 9.1: Falhas típicas e ações corretivas

Sintoma	Ação
GUI sem dados	Verificar CT3 (rede) e firewall; checar IP e porta no <code>UDPClient</code> .
Sensor sempre em HIGH	Ajustar sensibilidade do módulo SW-420 (trimpot) ou verificar ruído elétrico.
Muitos alertas falsos	Diminuir ganho do sensor; melhorar fixação mecânica; testar isolamento.
CSV vazio	Conferir permissões no diretório e função <code>export_to_csv()</code> .

9.9 Rastreabilidade de Requisitos

Tabela 9.2: Matriz de rastreabilidade (requisito → teste)

Requisito	Teste associado
Leitura digital funcional	CT1, CT2
Envio UDP conforme especificação	CT3, CT4
GUI em tempo real	CT5
Exportação de dados	CT6
Estabilidade e ausência de falsos positivos	CT7

9.10 Conclusão do Capítulo

Os testes confirmaram o funcionamento ponta a ponta do sistema, assegurando a leitura digital correta, a comunicação UDP estável e a visualização confiável na GUI. A exportação de relatórios e o desempenho estável demonstram que o projeto atende plenamente aos requisitos funcionais e de confiabilidade estabelecidos.

Capítulo 10

Conclusão

10.1 Resumo do Projeto

O projeto **Monitoramento Inteligente de Carga** cumpriu seu objetivo principal ao implementar um sistema embarcado funcional para detecção de vibrações utilizando o sensor **SW-420** no kit **STM32MP1-DK1**. O sistema realiza leituras digitais via GPIO, transmite os dados por **UDP** e exibe as informações em tempo real em uma interface gráfica desenvolvida em **PyQt5**.

Durante o desenvolvimento, foram aplicados conceitos essenciais de sistemas embarcados, comunicação em rede e integração entre C++ e Python, resultando em uma solução leve, modular e reprodutível.

10.2 Resultados e Validação

Os testes confirmaram o desempenho esperado:

- Leitura digital confiável do sensor SW-420;
- Transmissão UDP estável e de baixa latência;
- Exibição contínua e sem travamentos na GUI;
- Exportação de dados e cálculo de métricas em tempo real;
- Detecção precisa de impactos, sem falsos positivos em repouso.

A arquitetura modular do sistema garantiu facilidade de manutenção, clareza de código e documentação consistente.

10.3 Aprendizados e Aplicações

O projeto consolidou conhecimentos práticos em:

- Programação orientada a objetos em C++;
- Comunicação de rede via sockets UDP;
- Desenvolvimento de interfaces gráficas em Python (PyQt5);
- Integração entre hardware e software embarcado.

Além de seu valor didático, o sistema demonstra o potencial de sensores digitais simples aplicados a contextos de monitoramento e automação.

10.4 Considerações Finais

O trabalho evidencia a importância da integração entre **programação, eletrônica e comunicação de dados**. O sistema desenvolvido é funcional, robusto e escalável — uma base sólida para futuras aplicações em instrumentação, IoT e controle embarcado.

Em síntese, o projeto alcançou todos os objetivos propostos, mostrando que soluções acessíveis podem gerar resultados precisos e eficientes quando bem integradas.